😺 Robot Catty

Единая платформа управления робототехническими комплексами

```
╔═══════════════════════════════════════════════════╗
║                                                   ║
║   Одна консоль — тысячи машин                     ║
║   Один сервер — полный контроль                   ║
║                                                   ║
╚═══════════════════════════════════════════════════╝
```

[![Platform](https://img.shields.io/badge/Platform-Distributed-blue.svg)](https://) [![Uptime](https://img.shields.io/badge/Uptime-99.99%25-brightgreen.svg)](https://) [![License](https://img.shields.io/badge/License-Proprietary-darkblue.svg)](https://) [![Stack](https://img.shields.io/badge/Stack-Node.js%20%2B%20Angular-orange.svg)](https://)

📋 Концепция

Robot Catty — это серверная платформа, которая объединяет любые роботизированные устройства в единую сеть с централизованным управлением. Один сервер. Любое количество агентов. Полный контроль над каждым движением.

Система спроектирована для работы в условиях, когда надёжность и скорость имеют значение. Разработана для инженеров, которые привыкли контролировать всё до миллиметра.

🎯 Возможности платформы

1️⃣ Единый центр управления

Все подключённые устройства отображаются в одной консоли. Состояние каждого — в реальном времени.

Элемент консоли	Отображает
🗺️ Карта подключённых агентов	Список всех устройств, их статус (online / offline / error)
📊 Панель телеметрии	Температура, напряжение, загрузка, ошибки
⌨️ Командная строка	Отправка команд выбранному агенту или группе
📜 Лог событий	Хронология всех действий с метками времени
📈 Графики нагрузки	CPU, RAM, сеть, температура — на каждом узле

2️⃣ Групповое управление

Устройства можно объединять в группы и отдавать команды сразу всей группе.

Режим управления	Описание
🎯 Одиночный	Команда одному конкретному агенту
👥 Групповой	Команда всем агентам в выбранной группе
📢 Широковещательный	Команда всем подключённым устройствам
⛓️ Каскадный	Команда распространяется по цепочке: агент → агент → агент
⚡ Триггерный	Команда выполняется при наступлении условия (температура, время, событие)

3️⃣ Иерархия доступа

Система разграничивает права операторов. Каждый видит и может делать только то, что разрешено.

```
🔹 Уровень 0 — Наблюдатель
   ├── Видит статусы всех устройств
   ├── Читает логи
   └── Не может отправлять команды

🔹 Уровень 1 — Оператор
   ├── Управление устройствами в своей группе
   ├── Просмотр телеметрии
   └── Запуск утверждённых сценариев

🔹 Уровень 2 — Инженер
   ├── Конфигурирование агентов
   ├── Обновление прошивок
   ├── Изменение параметров подключения
   └── Создание новых групп

🔹 Уровень 3 — Администратор
   ├── Полный доступ ко всем устройствам
   ├── Назначение прав операторам
   ├── Изменение системных параметров
   └── Просмотр полного аудита

🔹 Уровень 4 — Главный пульт
   ├── Абсолютный контроль
   ├── Отмена любых команд
   └── Автономный режим поверх всех уровней
```

4️⃣ Автономность агентов

Каждое устройство может работать без связи с сервером по заложенной программе.

Режим	Условие	Поведение
🟢 Online	Связь есть	Полное управление с сервера
🟡 Offline (ожидание)	Нет связи < 10 с	Удержание последней позиции
🟠 Offline (автономия)	Нет связи > 10 с	Исполнение кэшированной программы
🔴 Offline (возврат)	Нет связи > 60 с	Последовательное завершение задач
⚫ Offline (авария)	Критическая ошибка	Остановка, сигнал маяка

При восстановлении связи — автоматическая синхронизация состояния и передача накопленных данных телеметрии.

5️⃣ Сценарии и автоматизация

Можно создавать последовательности действий, которые выполняются автоматически.

```
📋 Пример: «Ночной патруль»

1. ⏱️ 23:00 — активировать всех агентов группы «Охрана»
2. 🔄 Выполнить калибровку датчиков
3. 🚶 Начать движение по маршруту №1
4. 📡 Если датчик движения сработал — включить запись
5. 🔔 Если датчик движения сработал 3 раза — отправить уведомление
6. 🌅 06:00 — возврат всех агентов на базовые точки
7. 🔋 Переход в режим ожидания с подзарядкой
```

Сценарии можно составлять из блоков: движение, ожидание, проверка условия, цикл, уведомление.

6️⃣ Протоколы связи

Каждый агент поддерживает несколько каналов связи с автоматическим переключением.

Канал	⏱️ Задержка	📡 Дальность	Назначение
🔌 Проводной (UART/Serial)	< 1 мс	Кабель	Основное управление, гарантированная доставка
📶 WiFi (TCP/UDP)	< 10 мс	До 100 м	Оперативное управление
🔗 Mesh-сеть	< 50 мс	До 1 км	Роевое взаимодействие
📡 Дальняя связь	< 500 мс	Любая	Мониторинг и резерв

7️⃣ Журналирование и аудит

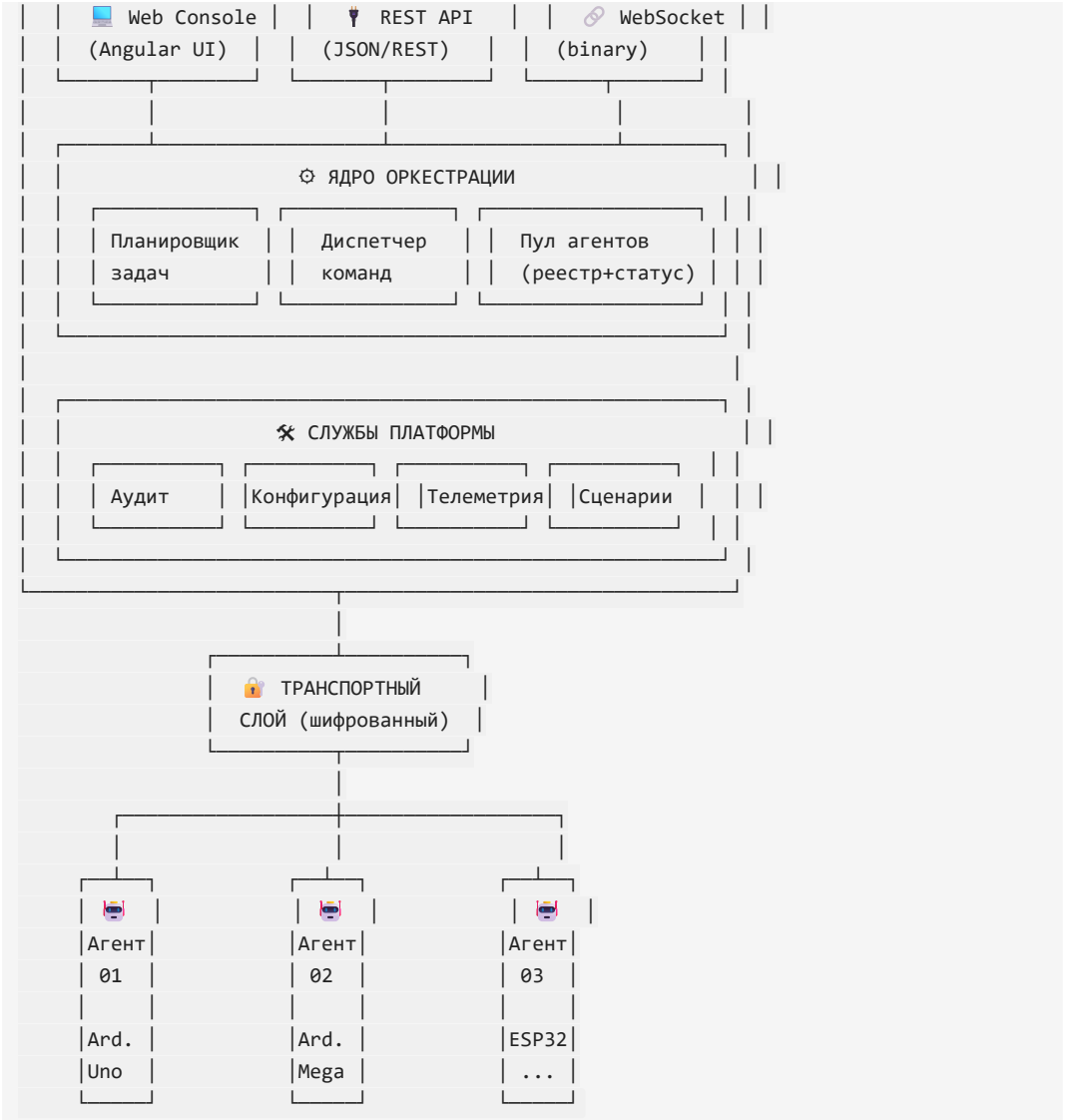
Всё, что происходит в системе, записывается и хранится.

Что логируется	Детали
📤 Команды	Кто, когда, кому, что отправил
📥 Ответы агентов	Успех/ошибка, время исполнения, результат
⚙️ Изменения конфигурации	Старое значение → новое, кто изменил
🔌 Подключения/отключения	Когда агент появился или пропал
❌ Ошибки и исключения	Тип, время, контекст, частота
📊 Метрики	Слепки состояния всех устройств (раз в секунду)

Срок хранения: индивидуально. Логи нельзя изменить или удалить задним числом.

🏗️ Архитектура

```
┌──────────────────────────────────────────────────────┐
│                    🧠 ОРКЕСТРАТОР                    │
│                  (Robot Catty Server)                │
│                                                      │
│  ┌──────────────┐  ┌──────────────┐  ┌─────────────┐ │
```



📊 Технические характеристики

Параметр	Показатель
🤖 Максимальное количество агентов	10 000+
⏱️ Частота опроса телеметрии	до 100 Гц
⚡ Задержка команды (локально)	< 5 мс
📶 Задержка команды (WiFi)	< 20 мс
🔐 Протокол шифрования	ECDHE + AES-256-GCM
📈 Аптайм (целевой)	99.99%
🚀 Время развёртывания нового агента	< 30 секунд
💾 Объём хранимого аудита	от 1 года
🧩 Платформы агентов	AVR, ARM, ESP32, RP2040, Linux
🖥️ Серверная часть	Node.js, TypeScript, PostgreSQL, Redis
🌐 Веб-интерфейс	Angular, WebSocket, Material Design
📖 Документация API	Swagger / OpenAPI

📱 Интерфейс

Веб-консоль доступна из любого браузера. Адаптивная вёрстка — работает на компьютере, планшете, телефоне.

Экран	Что показывает
🏠 Главная	Все агенты на одной панели, статус, быстрые команды
🎮 Управление	Слайдеры, кнопки, ползунки для каждого агента
📊 Телеметрия	Графики температуры, нагрузки, напряжения в реальном времени
📋 Сценарии	Конструктор последовательностей, библиотека готовых
📜 Логи	Поиск, фильтр, экспорт
⚙ Администрирование	Пользователи, роли, группы, настройки

🔌 Что можно подключить

Тип устройства	Пример
🦾 Сервоприводы	Движение, захват, поворот
⚙ Шаговые двигатели	Позиционирование, линейные перемещения
💡 RGB-индикаторы	Световая сигнализация
📡 Датчики	Температура, расстояние, освещённость, движение
📷 Камеры	Визуальный контроль (планируется)
🔭 3D-сенсоры	Глубина, расстояние, объём (планируется)
🔧 Исполнительные механизмы	Любые устройства с моторикой

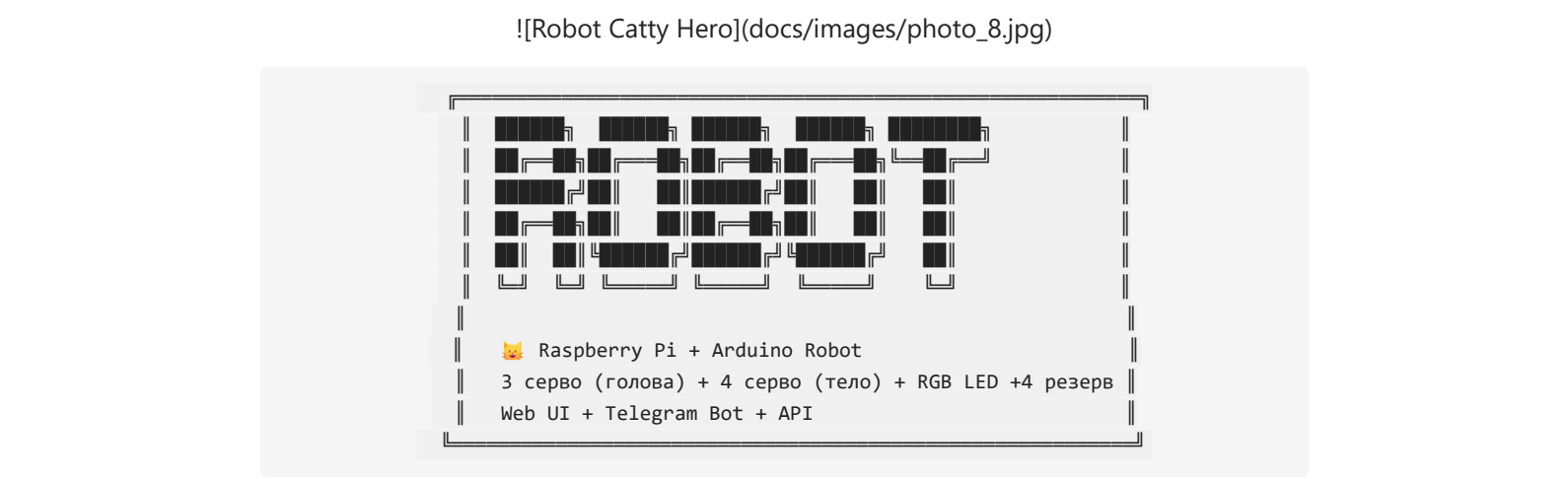
💡 Для кого эта платформа

Для инженеров, которые управляют техникой. Для тех, кому нужен единый пульт, а не десяток разных экранов. Для тех, кто строит автоматизированные системы любой сложности.

Всё в одном месте. Всё под контролем. Всё для своих.

> *Технологии не имеют идеологии. Идеологию им придают люди. Наша технология — для сохранения и развития.*

🤖 Robot Catty — техническая документация



[![License: MIT](https://img.shields.io/badge/License-MIT-yellow.svg)](https://opensource.org/licenses/MIT) [![Platform](https://img.shields.io/badge/Platform-Raspberry%20Pi%20%2B%20Arduino-blue.svg)](https://www.arduino.cc/)

Raspberry Pi + Arduino робот с веб-интерфейсом управления, Telegram ботом и REST API.

📋 О проекте

Robot Catty — это настольный робот на базе Raspberry Pi и двух Arduino, управляемый через веб-интерфейс, WebSocket и Telegram. Проект объединяет железо (сервоприводы, RGB-подсветка, камера) и софт (Node.js сервер, Angular UI, Capacitor Android-приложение, Telegram бот) в единую систему.

🎯 Возможности

Область	Что умеет
🎛️ **Управление**	7 сервоприводов (3 голова + 4 тело) + 4 резервных через слайдеры в реальном времени
💡 **RGB LED**	10 пресетов цветов, плавные переходы (FADE), мигание, ручные R/G/B слайдеры
🎬 **Анимации**	wave, nod, shake, dance, lookAround, blink — готовые сценарии движений
🌐 **Веб-интерфейс**	Angular SPA с WebSocket, статус в реальном времени, лог команд
🤖 **Telegram бот**	Управление RGB LED через команды: `/red`, `/blue`, `/color 255 128 0` и др.
📲 **Android-приложение**	Capacitor + PWA — мобильное управление с телефона
📡 **API**	REST + WebSocket + Swagger-документация
🧠 **CV (планируется)**	USB-камера + Xbox Kinect для компьютерного зрения
📊 **Мониторинг**	Системный мониторинг (CPU, RAM, температура) через веб-интерфейс
🔄 **Автозапуск**	Демо-режим при старте системы, state persistence

🧱 Стек технологий

```
┌─ Железо ──────────────────────────────────┐
│  Raspberry Pi 3B        — мозг, сервер    │
│  Arduino Uno (CH340)    — голова (3 серво)│
│  Arduino Mega 2560 R3   — тело (4+4 серво)│
│  Сервоприводы ×7        — движения        │
│  RGB LED                — глаза (индикация)│
│  USB Camera / Kinect    — зрение (CV)     │
└───────────────────────────────────────────┘

┌─ Софт ────────────────────────────────────┐
│  Node.js / Express      — веб-сервер      │
│  ws (WebSocket)         — real-time связь │
│  SerialPort             — связь с Arduino │
│  Angular SPA            — веб-интерфейс   │
│  Capacitor              — Android-обёртка │
│  Telegram Bot (Python)  — управление      │
│  Swagger / OpenAPI      — документация    │
└───────────────────────────────────────────┘
```

💡 Для кого это

- **DIY-энтузиасты** — готовый open-source проект робота от идеи до работающего прототипа
- **Разработчики** — пример интеграции Arduino + Raspberry Pi + Node.js + WebSocket + Angular
- **Изучающие робототехнику** — Reference Implementation с полной документацией (схемы, распиновка, wiring)

📑 Оглавление

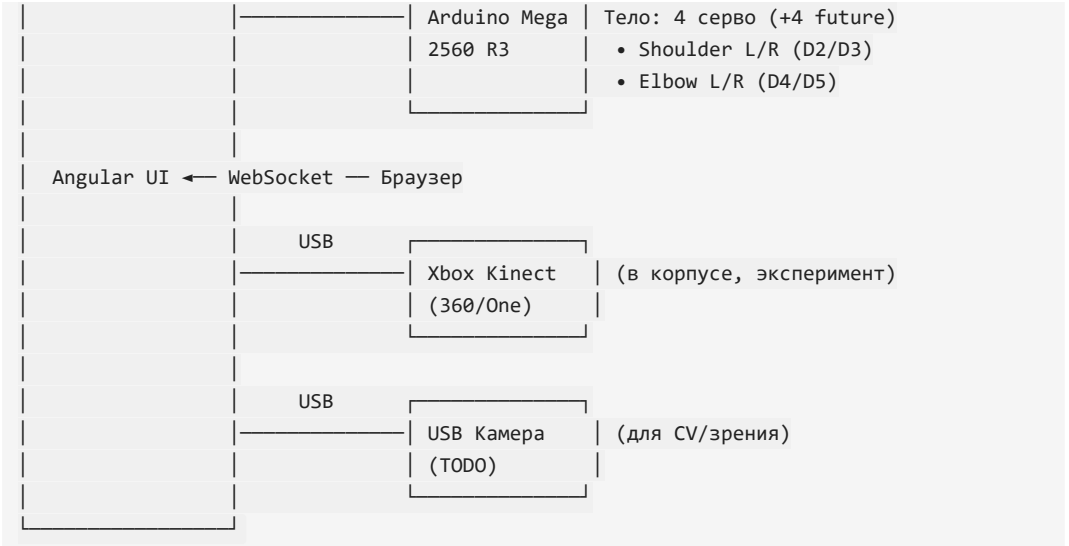
- 🏗️ Архитектура
- ⚡ Подключение и питание
- Камера и Kinect
- 🖥️ Управление и API
- 🚀 Установка и запуск
- 📁 Структура проекта
- ⚠️ Известные проблемы
- 📸 Галерея
- 📝 Блог и лицензия

🏗️ Архитектура

 Robot Catty Architecture

Общая схема

```
┌──────────────────┐          USB          ┌──────────────┐
│                  │◄─────────────────────►│ Arduino Uno  │ Голова: 3 серво + RGB LED
│  Raspberry Pi    │                       │ (CH340)      │  • Левый глаз (D4)
│  Node.js :3000   │                       │              │  • Правый глаз (D7)
│                  │                       │              │  • Челюсть (D8)
│                  │                       └──────────────┘
│                  │          USB          ┌──────────────┐
│                  │◄─────────────────────►│ Arduino Mega │ Тело: 4+4 серво
│                  │                       │ 2560 R3      │  • Левая рука (D2, D3)
│                  │                       │              │  • Правая рука (D5, D6)
│                  │                       │              │  • Левая нога (D9, D10)
│                  │                       │              │  • Правая нога (D11, D12)
└──────────────────┘                       └──────────────┘
```



Компоненты

Компонент	Роль	Подключение
Raspberry Pi 3B	Контроллер, веб-сервер	Ethernet/WiFi
Arduino Uno (CH340)	Голова: 3 серво + RGB LED	USB → /dev/ttyUSB0
Arduino Mega 2560 R3	Тело: 4 серво (+4 резерв)	USB → /dev/ttyACM0
RGB LED (4-pin)	Индикация глаз	Uno D3/D5/D6
Сервоприводы (×7)	Движение	Uno D4,D7,D8 / Mega D2-D5
USB Camera	Компьютерное зрение (CV)	USB (порт освободится)
Xbox Kinect (360/One)	3D-сенсор, трекинг, CV	USB (в корпусе, треб. свободный порт)

Распиновка

Arduino Uno (Голова)

Пин	Компонент	Описание
D3	RGB R	Красный светодиод (через 220Ω)
D5	RGB G	Зелёный светодиод (через 220Ω)
D6	RGB B	Синий светодиод (через 220Ω)
D4	Servo eyeLR	Левый/правый глаз ↔
D7	Servo eyeUD	Глаза вверх/вниз ↕
D8	Servo jaw	Челюсть (открыть/закрыть)
GND	RGB GND	Общий провод

Arduino Mega 2560 (Тело)

Пин	Компонент	Описание
D2	Servo	Левое плечо
D3	Servo	Правое плечо
D4	Servo	Левый локоть
D5	Servo	Правый локоть
D6-D9	Servo	Резерв (future)

> ⚠️ НЕ питать сервоприводы от Arduino 5V! Используйте отдельный блок 5-6V, минимум 2A на серво. GND всех источников соединить вместе.

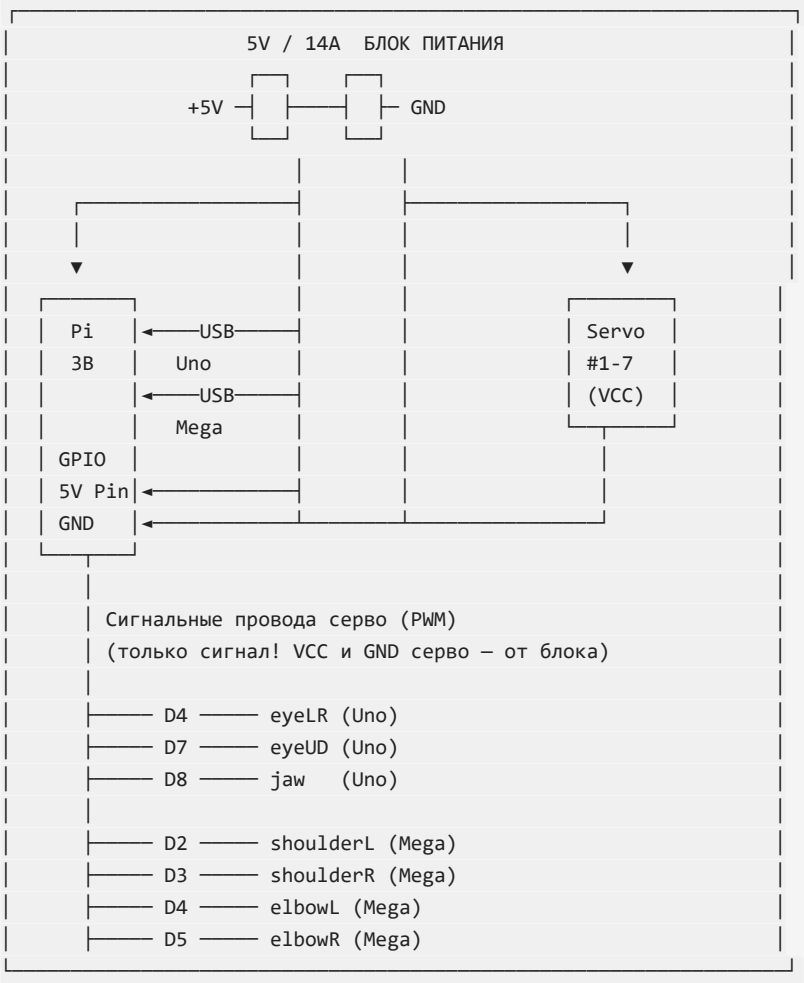
Интерактивные схемы

- 🔗 Полная интерактивная схема — кликабельная SVG с подробными подписями
- 📄 Wiring документация — полное описание подключения

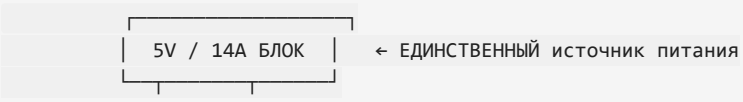
⚡ Подключение и питание

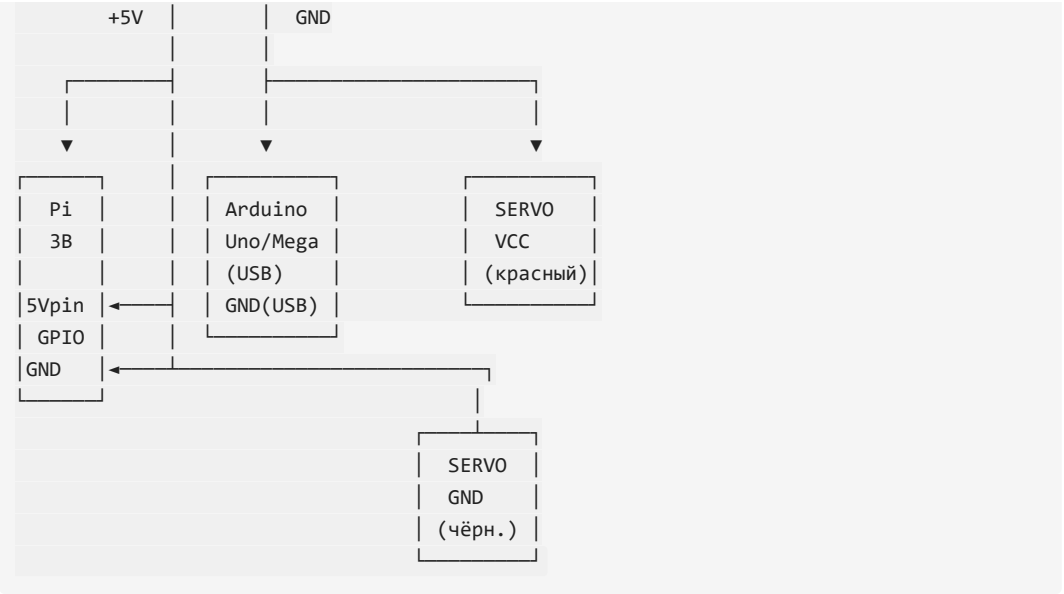
> ⚠️ **ВАЖНОЕ ИЗМЕНЕНИЕ**: Вся система питается от **одного блока 5V**. Никакого 12V! Ниже — правильная схема с раздельным питанием сервоприводов.

Схема подключения



Детальная схема питания





Правила подключения (пошагово)

1️⃣ Питание сервоприводов

У каждого сервопривода **3 провода**:

Цвет	Назначение	Куда подключить
🔴 Красный	VCC (+5V)	**Напрямую к +5V блока питания**
⚫ Чёрный/Коричневый	GND	**Напрямую к GND блока питания**
🟡 Оранжевый/Белый	Сигнал (PWM)	К пину Arduino (см. таблицу)

⚠️ **Ни в коем случае не подключай VCC серво к Arduino или Pi!** Только сигнальный провод.

2️⃣ Питание Raspberry Pi

- **5V** — GPIO пин **#2** или **#4** (физические) → к **+5V блока**
- **GND** — GPIO пин **#6**, **#9**, **#14**, **#39** → к **GND блока**

Pi 3B возьмёт свои ~1.5A из общих 14A — с запасом.

3️⃣ Питание Arduino

- **Обе Arduino питаются по USB от Raspberry Pi**
- USB даёт и данные (Serial) и питание (5V)
- Через USB GND Arduino уже соединён с GND Pi

4️⃣ Общий GND (критически важно!)

GND должны быть соединены ВСЕ вместе — иначе серво будут дёргаться, а управление — сбоить:

```
GND блока питания ── GND Pi (GPIO) ── GND Arduino (через USB) ── GND серво
```

Это единая "земля" для всей системы. Без этого сигнал PWM не работает стабильно.

5️⃣ Сигнальные провода серво → Arduino

Только сигнал! VCC и GND серво уже подключены к блоку.

Arduino Uno (Голова):

Пин	Сигнал серво	Серво
D4	🟡 к eyeLR	Левый/правый глаз
D7	🟡 к eyeUD	Глаза вверх/вниз
D8	🟡 к jaw	Челюсть

Arduino Mega (Тело):

Пин	Сигнал серво	Серво
D2	🟡 к shoulderL	Левое плечо
D3	🟡 к shoulderR	Правое плечо
D4	🟡 к elbowL	Левый локоть
D5	🟡 к elbowR	Правый локоть
D6-D9	🟡 (резерв)	Будущие серво

6️⃣ RGB LED

4-pin RGB LED (общий катод) — питается от Arduino Uno через резисторы:

Цвет	Пин Uno	Резистор
🔴 R	D3 (PWM)	220Ω
🟢 G	D5 (PWM)	220Ω
🔵 B	D6 (PWM)	220Ω
⚫ GND	GND	—

7️⃣ Камера (USB) и Xbox Kinect

USB-камера (веб-камера, например Logitech C270/C920): - Планируется для компьютерного зрения (распознавание жестов, лиц, объектов) - USB-порт: необходимо освободить один USB-порт на Pi (возможно отключение WiFi-адаптера, т.к. Pi на проводном Ethernet) - После подключения: `/dev/video0`, тест через `fswebcam` или `v4l2-ctl` - Пример: `fswebcam -d /dev/video0 -r 640x480 test.jpg`

Xbox Kinect (360 или One, уже смонтирован в корпус робота): - 3D-сенсор для компьютерного зрения: глубина, скелетный трекинг, RGB - **Требования:** - Свободный USB-порт + дополнительное питание (если через адаптер) - На Pi: библиотека `libfreenect` (для Kinect 360) или `libfreenect2` (для Kinect One) - Большая нагрузка на CPU — Pi 3B может не тянуть полноценный скелетный трекинг (>60% CPU) - **Возможное применение:** детекция человека перед роботом, трекинг движения, управление жестами - **Статус:** эксперимент — активная настройка отложена до проверки нагрузки на Pi

> ⚠️ На Pi 3B Kinect 360 даёт ~5-8 FPS RGB + depth. Для полноценного CV рекомендуется внешний сервер (OpenCV на мощной машине, Pi — только трансляция).

Расчёт потребления

Компонент	Ток (средний)	Ток (пик)
Raspberry Pi 3B	~1.0A	~1.5A
Arduino Uno	~0.05A	~0.1A
Arduino Mega	~0.1A	~0.2A
1 серво (движение)	~0.3A	~0.8A
1 серво (удержание)	~0.1A	~0.2A
7 серво (движение)	**~2.1A**	**~5.6A**
Итого с запасом	**~3.5A**	**~7.5A**

Твой блок **5V 14A** — более чем достаточен. Запас по току 2x — идеально.

⛔ Чего НЕ делать

- ❌ Не питать серво через Arduino 5V — сожжёшь стабилизатор
- ❌ Не питать серво через Pi 5V — дорожки не рассчитаны, будут просадки и зависания
- ❌ Не использовать 12V — серво и Pi работают от 5V, 12V спалит всё
- ❌ Не соединять GND через один тонкий провод — используй общую шину (толстый провод или общую точку)
- ❌ Не подключать VCC серво к сигнальным пинам Arduino — сожжёшь порт

🛠️ Порядок сборки (рекомендуемый)

1. **Подключи блок питания** — пока ничего не включай
2. **Соедини GND всех устройств** — общая шина
3. **Подключи VCC серво к +5V блока** — все красные провода вместе
4. **Подключи GND серво к GND блока** — все чёрные провода вместе
5. **Подключи сигналы серво к Arduino** — каждый оранжевый провод к своему пину
6. **Подключи Pi 5V к +5V блока** — через GPIO
7. **Подключи Arduino к Pi по USB** — данные + питание ардуин
8. **Подключи RGB LED к Uno** — через резисторы 220Ω
9. **Включи блок питания** — система запустится

💻 Управление и API

Serial протокол (9600 baud)

Arduino Uno (Голова)

Команда	Описание	Пример
`SERVO:<name>:<angle>`	Установить угол серво (0-180)	`SERVO:eyeL:120`
`RGB:<r>,<g>,<b>`	Установить цвет (0-255)	`RGB:255,128,0`
`OFF`	Выключить RGB	`OFF`
`FADE:<r>,<g>,<b>`	Плавный переход к цвету	`FADE:0,0,255`
`BLINK`	Режим мигания	`BLINK`

Команда	Описание	Пример
`PRESET:<name>`	Готовый цвет	`PRESET:red`
`STATUS`	Текущее состояние	`STATUS`

Arduino Mega (Тело)

Команда	Описание	Пример
`SERVO:<name>:<angle>`	Установить угол серво (0-180)	`SERVO:shoulderL:120`
`CENTER`	Центрировать все серво	`CENTER`
`STATUS`	Текущее состояние	`STATUS`

Пресеты цветов

`red`, `green`, `blue`, `white`, `yellow`, `cyan`, `magenta`, `orange`, `pink`, `purple`

REST API

Метод	Путь	Описание
GET	`/api/status`	Статус подключения + состояние
POST	`/api/servo`	`{board, servo, angle}`
POST	`/api/rgb`	`{r, g, b}`
POST	`/api/rgb/off`	Выключить RGB
POST	`/api/rgb/preset`	`{name}`
POST	`/api/animation`	`{name, params}`
POST	`/api/animation/stop`	Остановить анимацию

WebSocket

Подключение: - **Через домен:** `wss://YOUR_DOMAIN/ws` (рекомендуется — работает через nginx с SSL) - **Прямой IP:** `ws://PI_IP:3000/ws` (только в локальной сети)

```
{"type": "servo", "board": "head", "servo": "eyeL", "angle": 120}
{"type": "rgb", "r": 255, "g": 0, "b": 0}
{"type": "preset", "name": "red"}
{"type": "animation", "name": "wave"}
{"type": "stop"}
```

В веб-интерфейсе доступен переключатель между режимами «Через домен» и «Прямой IP».

Веб-интерфейс

```
cd ~/robot-catty/server
npm start
# Открыть http://<pi-ip>:3000
```

Возможности UI: - 🎛️ Управление серво — слайдеры для всех серво - 💡 RGB LED — слайдеры R/G/B + 10 пресетов + кнопка выключения - 🎬 Анимации: `wave`, `nod`, `shake`, `dance`, `lookAround`, `blink` - 📊 Статус в реальном времени — WebSocket - 📋 Лог — все команды и события

Telegram бот

Управление RGB LED через Telegram.

Команды: `/red`, `/green`, `/blue`, `/white`, `/yellow`, `/cyan`, `/magenta`, `/orange`, `/pink`, `/purple`, `/off`, `/status`, `/blink`, `/color R G B`, `/demo`, `/stop`

> ⚠️ Токен бота нужно запросить у @BotFather и вставить в `scripts/rgb_bot.py`

🚀 Установка и запуск

Raspberry Pi

```
# Клонирование
git clone https://github.com/RuslanStrogov/robot-catty.git
cd robot-catty

# Установка Node.js зависимостей
cd server
npm install

# Запуск
npm start

# Автозапуск (crontab)
crontab -e
# Добавить:
# @reboot cd /path/to/robot-catty/server && /usr/bin/node server.js
```

Загрузка скетчей

```
# Arduino Uno (голова — 3 серво + RGB)
avr-gcc -mmcu=atmega328p -DF_CPU=16000000L \
  -DARDUINO=10607 -DARDUINO_AVR_UNO -DARDUINO_ARCH_AVR \
  -I$CORE -I$VAR \
  -Os -w -std=gnu++11 -fpermissive -fno-exceptions \
  -ffunction-sections -fdata-sections -fno-threadsafe-statics -flto \
  $CORE/*.cpp $CORE/*.c \
  firmware/servo_head/servo_head.cpp -o servo_head.elf -lm

avrdude -c arduino -p m328p -P /dev/ttyUSB0 -b 115200 -D -U flash:w:servo_head.hex:i

# Arduino Mega (тело)
arduino-cli compile -b arduino:avr:mega firmware/servo_body/
arduino-cli upload -b arduino:avr:mega -p /dev/ttyACM0 firmware/servo_body/
```

Демо режим

Автоматический запуск при старте системы (crontab `@reboot`):

1. 🔴 Моргание красным — 1 минута
2. Каждый пресет — 3 минуты с плавным переходом

📁 Структура проекта

```
robot-catty/
├── firmware/
│   ├── servo_head/servo_head.cpp    # Uno — голова (3 серво + RGB)
│   └── servo_body/servo_body.ino    # Mega — тело (4 серво)
├── server/
│   ├── package.json                 # Node.js зависимости
│   ├── server.js                    # Express + WebSocket сервер
│   ├── services/
│   │   ├── arduino.js               # Serial связь с Arduino
│   │   └── robot.js                 # Логика робота + анимации
│   └── public/
│       └── index.html               # Angular веб-интерфейс
├── config/
│   ├── arduino_config.md            # Настройки плат
│   └── servo_config.json            # Конфигурация серво (пины, лимиты)
├── docs/
│   ├── wiring.md                    # Схема подключения (полная)
│   ├── interactive-schematics.md    # Описание интерактивных схем
│   ├── robot-catty-circuit.html     # Интерактивная SVG
│   └── images/                      # Фото и схемы
├── scripts/
│   ├── rgb_bot.py                   # Telegram бот управления RGB
│   └── demo.py                      # Демо-режим (автозапуск)
├── remote/
│   └── rgb_led_remote.py            # Удалённое управление с Windows
├── ISSUES.md                        # Известные задачи и проблемы
├── CREATE_ISSUES.md                 # Шаблоны для GitHub Issues
└── README.md                        # Этот файл
```

⚠️ Известные проблемы

- Arduino Uno CH340: все фюзы = 0x00 (125kHz клок), но serial работает на 9600 baud
- Для компиляции на Pi нужен Arduino IDE или PlatformIO (версия atmelavr ≤5.3.0 для корректной работы с Mega 2560)
- AVR core хранится в `/tmp/` или `~/.platformio/` — теряется после перезагрузки при использовании Snap
- Mega 2560: Servo library конфликтует с Timer1 — используйте прямую ШИМ или `Servo.h` с `lib_deps`
- **USB-камера:** не подключена — все USB-порты заняты (WiFi-адаптер TP-Link и 2× Arduino). Решение: отключить WiFi при проводном Ethernet
- **Xbox Kinect:** смонтирован в корпус, но активная настройка отложена. Pi 3B может не тянуть скелетный трекинг (>60% CPU на libfreenect). Возможно потребуется внешний сервер для OpenCV
- **WebSocket:** при подключении через прямой IP (`ws://IP:3000`) может быть ошибка `WebSocket connection failed` из-за отсутствия SSL — используйте `wss://YOUR_DOMAIN/ws` когда сайт открыт через HTTPS

Подробнее в ISSUES.md

📸 Галерея

Главный интерфейс



Возможности UI

- 🎛️ **Управление серво** — слайдеры для всех серво (голова + тело)
- 💡 **RGB LED** — слайдеры R/G/B + 10 пресетов + кнопка выключения
- 🎬 **Анимации** — `wave`, `nod`, `shake`, `dance`, `lookAround`, `blink`
- 📊 **Статус в реальном времени** — WebSocket подключение к Arduino
- 📋 **Лог** — все команды и события

Фото прототипов

| Прототип 1 | Прототип 2 | Прототип 3 | |:---:|:---:|:---:| | ![UI Prototype 1](docs/images/ui-prototypes/proto_1.jpg) | ![UI Prototype 2](docs/images/ui-prototypes/proto_2.jpg) | ![UI Prototype 3](docs/images/ui-prototypes/proto_3.jpg) |

📝 Блог и лицензия

- **Telegram**: http://t.me/itvpi
- **VK**: https://vk.com/itvpiska

Лицензия

MIT